RESEARCH-ARTICLE

To Mock or Not to Mock: Divergence in Mocking Practices Between LLM and Developers

HANBIN QIN, Stevens Institute of Technology, Hoboken, NJ, United States

Published: 13 June 2025

2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)
March 27 - April 3, 2025

https://doi.org/10.1109/ICSE-Companion66252.2025.00077
EISBN: 979-8-3315-3683-1

2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion) | 979-8-3315-3683-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICSE-COMPANION66252.2025.00077

To Mock or Not to Mock: Divergence in Mocking Practices Between LLM and Developers

Hanbin Qin
School of Engineering and Science
Stevens Institute of Technology
Hoboken, New Jersey, USA
hqin5@stevens.edu

***Abstract*—Mock objects are essential for isolating unit tests and reducing dependencies in software testing. However, deciding what to mock requires careful judgment to balance isolation and maintainability. This study evaluates OpenAI's GPT-4o for automating mock decisions by comparing its outputs with developer choices. The findings reveal that while the LLM excels in identifying dependencies, their broader isolation strategy often results in Over-mocking compared to the developers. These insights suggest the potential for LLM-based tools to generate test cases with accurate and well-balanced mocking strategies.**

I. Introduction

Mock objects are used in unit testing to simulate the behavior of real objects, isolating the function under test from its dependencies [1] [2] [3]. Mocking decisions significantly affect test quality and maintainability—excessive mocking creates brittle tests, while insufficient mocking reduces isolation [4] [5] [6]. Developers make these decisions based on expertise, but the process is subjective and inconsistent [7] [8]. Large language models (LLMs) present a new avenue for automating such tasks, offering the promise of consistency and efficiency [9]. However, little is known about how well LLMs replicate developer judgment in mocking decisions or how their strategies align with best practices [10]. This research addresses these gaps by investigating the effectiveness of LLM, specifically OpenAI's GPT-4o [11] in generating mock objects and identifying areas where their approach diverges from developer strategies.

II. Study Approach

This study examines OpenAI's GPT-4o, a high-performing LLM, using Apache Dubbo [12], an actively maintained large Java open-source project, as a case study. The experiment involves 576 test suites and 1,482 test cases, following three key steps:

a) Test Suite Conversion to Mask Developer Decisions: Developer-created test suites were converted to obscure object instantiations, preserving structural integrity while anonymizing object creation logic. This step ensured that GPT-4o made independent decisions about which objects to mock.

b) Mock Generation Using converted Test Suite by LLM: The converted test suites were provided to GPT-4o, which analyzed them to determine appropriate mock objects. A zero-shot prompting approach was used to avoid bias [13], encouraging the LLM to apply general reasoning rather than specific pre-defined rules.

c) Comparison and Discrepancy Analysis: LLM-generated decisions were compared with developer choices using precision, recall, and F1-score. Discrepancies were categorized as Over-mocking or Under-mocking: Over-mocking represents the objects mocked by LLM but not mocked by developers, while Under-mocking stands for the objects not mocked by LLM but mocked by developers. Six widely-adopted mock object categories [7]—Database, Web Service, Domain Objects, Test Support, Native Java libraries, and External Dependencies [14] [15] [6]—were analyzed to identify patterns and trends.

This methodology is unique in its focus on a direct comparison between LLM-generated and developer-generated mock decisions. It illuminates LLM's ability to identify appropriate dependencies to mock in software testing.

III. Results

As a result, GPT-4o achieved a recall of 92%, indicating strong coverage of dependencies suitable for mocking. However, its precision was lower at 28%, leading to an F1-score of 43%. These results highlight GPT-4o's broad coverage of developer decisions but also a tendency to over-mock by introducing extra mocks not created by developers.

Figure 1 shows the distribution of Over-Mocking, Under-Mocking, and Matched-Mocking across six mock object types [7]. Alignment between the LLM and developers was highest for *Database* objects (100%) and lowest for *Test Support* (43.62%), which also had the highest over-mocking rate (54.26%). Under-Mocking is rare, with high recall and low precision, making it barely visible in the figure. Categories ranked by match percentage are: *Database* (100%), *Web Service* (73.51%), *Native Java Libraries* (71.78%), *Domain Objects* (64.10%), and *External Dependencies*. These results suggest GPT-4o aligns well in some cases but often adopts broader isolation, particularly for *Test Support*.

IV. Future Plan

Future work could focus on helping GPT-4o balance isolation and simplicity by distinguishing high-impact mocks and aligning them with developer practices. Adding context, such

2574-1934/25/$31.00 ©2025 IEEE
DOI 10.1109/ICSE-Companion66252.2025.00077

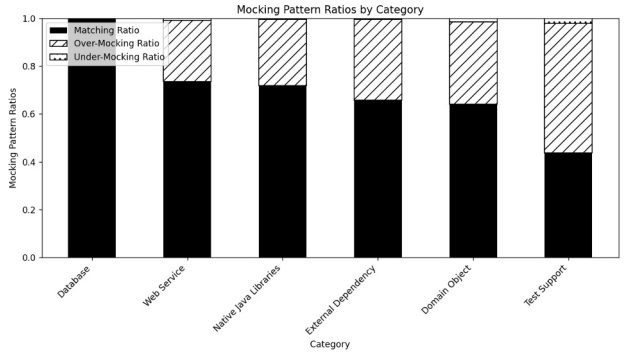

Fig. 1: Mocking Patterns by Category

as dependency interfaces or architectural details, could improve judgment and reduce Over-Mocking. Exploring domain-specific mocking needs and fine-tuning the model may further enhance its ability to make more context-sensitive decisions, better aligning with developer approaches for higher test quality.

V. Related Work

Spadini et al. [7] empirically categorized mocked and non-mocked dependencies, while Taneja et al. [2] and Zhu et al. [16] introduced tools for automated test generation and mocking recommendations. While many empirical studies have focused on mocks in testing, few have explored the best practices or evaluated existing ones.

Lemieux et al. [10], Pan et al. [17], and Zhang et al. [18] demonstrated the potential of LLMs in generating and improving test cases with mocks, affirming their promise as tools for automated test generation. However, research on mock generation using LLMs remains scarce.

VI. Conclusion

This study highlights the strengths and limitations of LLM, specifically, OpenAI's GPT-4o in mocking decisions for Java testing suites. As a result, GPT-4o's mocking decision aligns with developers on certain dependencies, but often favors broader isolation, leading to occasional Over-Mocking, while developers take a more selective approach. Future efforts could refine LLMs to distinguish essential mocks and adapt strategies for domain-specific codes, aligning them more closely with developer practices to enhance automated testing tools and improve test effectiveness.

Acknowledgment

This work was supported in part by the U.S. National Science Foundation (NSF) under grants CCF-2044888, CCF-1909763, and CCF-2348338.

References

[1] M. R. Marri, T. Xie, N. Tillmann, J. de Halleux, and W. Schulte, "An empirical study of testing file-system-dependent software with mock objects," in *2009 ICSE Workshop on Automation of Software Test*, 2009, pp. 149–153.

[2] K. Taneja, Y. Zhang, and T. Xie, "Moda: Automated test generation for database applications via mock objects," in *Proceedings of the IEEE/ACM international conference on Automated software engineering*, 2010, pp. 289–292.

[3] S. Mostafa and X. Wang, "An empirical study on the usage of mocking frameworks in software testing," in *2014 14th international conference on quality software*. IEEE, 2014, pp. 127–132.

[4] D. Spadini, M. Aniche, M. Bruntink, and A. Bacchelli, "Mock objects for testing java systems," *Empirical Software Engineering*, vol. 24, no. 3, pp. 1461–1498, 2019.

[5] N. Alshahwan, M. Harman, and A. Marginean, "Software testing research challenges: An industrial perspective," in *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2023, pp. 1–10.

[6] G. Pereira and A. Hora, "Assessing mock classes: An empirical study," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2020, pp. 453–463.

[7] D. Spadini, M. Aniche, M. Bruntink, and A. Bacchelli, "To mock or not to mock? an empirical study on mocking practices," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 2017, pp. 402–412.

[8] T. Kim, C. Park, and C. Wu, "Mock object models for test driven development," in *Fourth International Conference on Software Engineering Research, Management and Applications (SERA'06)*, 2006, pp. 221–228.

[9] T. Wu, S. He, J. Liu, S. Sun, K. Liu, Q.-L. Han, and Y. Tang, "A brief overview of chatgpt: The history, status quo and potential future development," *IEEE/CAA Journal of Automatica Sinica*, vol. 10, no. 5, pp. 1122–1136, 2023.

[10] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 919–931.

[11] OpenAI, "GPT-4o," https://openai.com/index/hello-gpt-4o/, 2024, [Online; accessed 10-Nov-2024].

[12] Apache Dubbo Team, "Apache Dubbo v3.2.16," https://github.com/apache/dubbo/releases/tag/dubbo-3.2.16, 2023, [Online; accessed 10-Nov-2024].

[13] L. Reynolds and K. McDonell, "Prompt programming for large language models: Beyond the few-shot paradigm," 2021. [Online]. Available: https://arxiv.org/abs/2102.07350

[14] E. Evans, *Domain-driven design: tackling complexity in the heart of software*. Addison-Wesley Professional, 2004.

[15] M. Fowler, *Patterns of enterprise application architecture*. Addison-Wesley Professional, 2002.

[16] H. Zhu, L. Wei, M. Wen, Y. Liu, S.-C. Cheung, Q. Sheng, and C. Zhou, "Mocksniffer: Characterizing and recommending mocking decisions for unit tests," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 2020, pp. 436–447.

[17] R. Pan, M. Kim, R. Krishna, R. Pavuluri, and S. Sinha, "Multi-language unit test generation using llms," 2024. [Online]. Available: https://arxiv.org/abs/2409.03093

[18] Z. Zhang, X. Liu, Y. Lin, X. Gao, H. Sun, and Y. Yuan, "Llm-based unit test generation via property retrieval," 2024. [Online]. Available: https://arxiv.org/abs/2410.13542